CS Most Influential People: Biographical Information

Hood College, IT 530, Fall 2025

5. Final Report

Group Name: Superstar

Team Members: Anthony Lombardi, Hamere Mogus, Han Soe, Jamilet Cordon

Instructor: Carol Jim

Submission Date: December 5, 2025

Table of Contents:

1. Topic Selection --Page 3
2. ER Design --Page 5
3. Scheme and SQL Implementation --Page 6
4. User Interface design --Page 16

I. Topic Selection:

DBMS for CS Most Influential People: Biographical Information

Description:

The Superstar Consulting Group (SCG), consisting of members Anthony Lombardi, Hamere Mogus, Han Soe, and Jamilet Cordon must configure an online repository of cybersecurity's most influential figures and their related works. Current databases do not allow for quick and efficient information gathering in regard to individuals who have contributed most to the field, often requiring cross-referencing multiple siloed sources. As cybersecurity continues to grow, there is an increased need for a one stop solution relating to the biographical information for the most influential contributors. A web-facing relational database system will provide users with the information necessary to learn more about cybersecurity as a whole. The system will manage Contributor Profiles, Key Publications, and Associated Organizations. This platform will reduce user research time from minutes to under 30 seconds, providing academic, governmental, and industry users with a verified, single source of truth to track expertise and historical context.

Scope:

The proposed system utilizes a relational database as an information repository. The database will contain biographical information for cybersecurity's most influential people. Current information gathering processes for historical and contemporary cybersecurity contributors are fragmented and inefficient. This centralized platform solves the problem by providing a verified, single source of truth designed for efficient knowledge discovery. The system utilizes a relational database model to me model to manage comprehensive biographical comprehensive biographical and technical information.

Data Entities:

Identification: This will include standard identifying information (contributor profiles) like name, nationality, and primary domains. Additionally, including unique identifiers for quick profile access.

Detailed Data (Context):

Flexible component captures related entities like related works and their URLs to access them, subfield expertise areas the individual specialized in, and links to bio information of the

referenced individuals. Management & Access: The system also captures necessary management control information to track system activities.

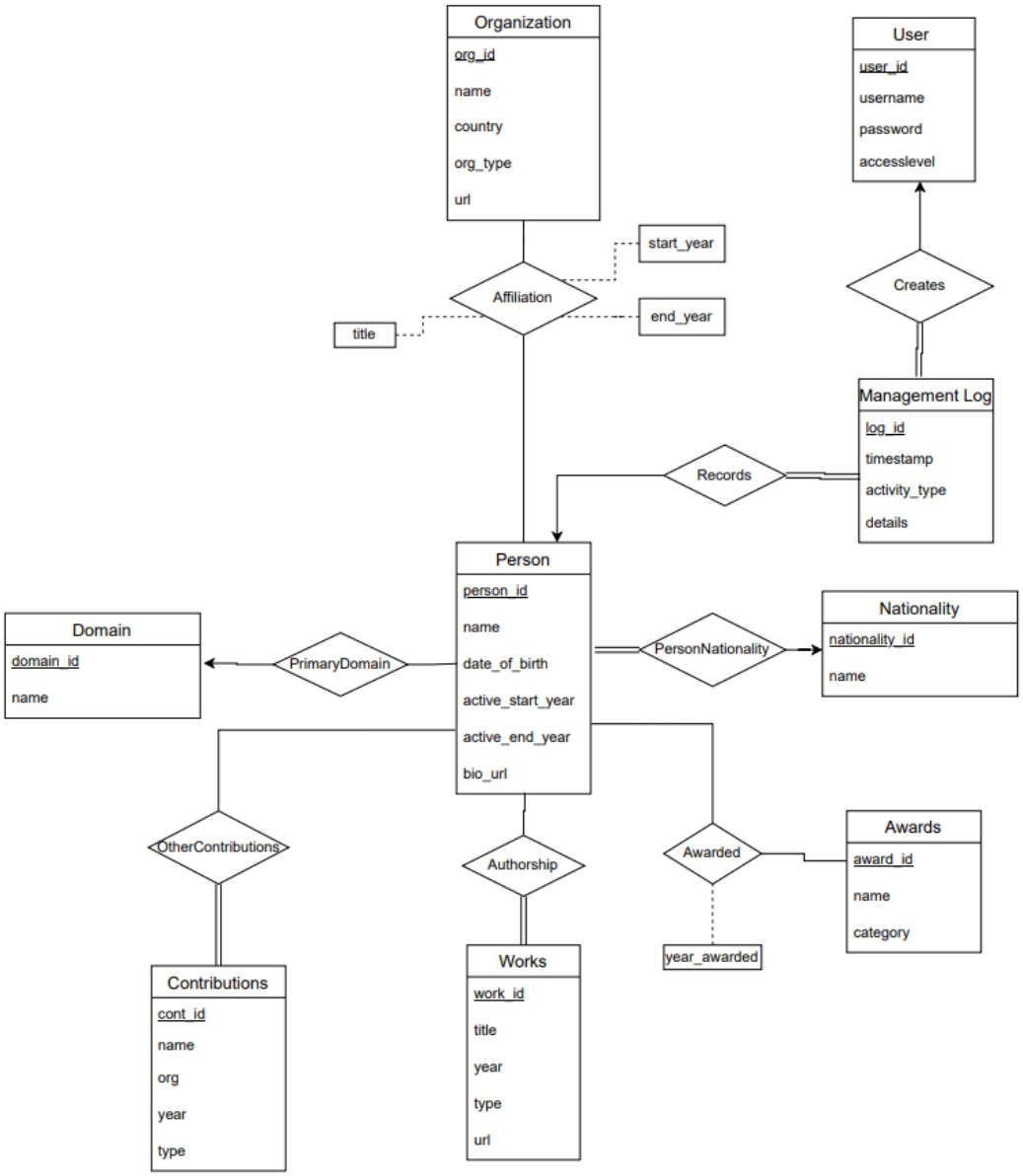
Data entry:

Dedicated data entry forms w/authenticated users. Data entry forms will also provide the option to add new individuals to the database including their information, works, biographies, etc. The database will allow web users to access the database.

Reporting &Visualization:

Data entry forms will provide the ability to add new information relating to existing figures. The user interface provides structured reports and visualizations.

II.ER Design:


Organization
org_id
name
country
org_type
url
User
user_id
username
password
accesslevel
start_year
Affiliation
end_year
title
Creates
Management Log
log_id
timestamp
activity_type
details
Records
Person
person_id
name
date_of_birth
active_start_year
active_end_year
bio_url
Domain
domain_id
name
PrimaryDomain
PersonNationality
Nationality
nationality_id
name
OtherContributions
Authorship
Awarded
Awards
award_id
name
category
year_awarded
Contributions
cont_id
name
org
year
type
Works
work_id
title
year
type
url

III. Schema and SQL Implementation:

Relational Schema:

1.1 PERSON (Han)

Person (id:INTEGER, name: VARCHAR (50), date_of_birth:DATE, active_start_year:INT(4), active_end_year:INT(4), bio_url:VARCHAR(300), domain id: INT, nationality id:INT)

The person table stores information about each person, such as their id, name and date of birth. The personr's id is the primary key for the person table.

1.2 ORGANIZATION (Han)

Organization (org id: INTEGER, name: VARCHAR(200), country:VARCHAR(50), org_type: VARCHAR(50), url:VARCHAR(300))

The Organization table contains the information about organization such as organization identification, name and country. Organization identification (id) is the primary key for the table.

1.3 CONTRIBUTION (Anthony)

Contributions (cont_id: INTEGER, name: VARCHAR (25), role: VARCHAR (20), org: VARCHAR (50), year: INTEGER, type: VARCHAR(20))

The contributions relation stores information relating to contributions within the CS field. It includes a contribution ID (primary key), the name, the role the individual played, the organization related to the contribution, the year, and the type of contribution.

1.4 DOMAINS (Anthony)

Domain (domain_id: INTEGER, name: VARCHAR (25))

The domain relation stores information about the domain of the individuals including a domain ID and a name.

1.5 AWARD (Hamere)

Award (award_id: INT, award_name: VARCHAR (300), category: VARCHAR(120))

Award catalogs distinct prizes and honors. Storing award name and optional category allows distinguishing similarly named awards or multiple categories.

1.5.1 PERSONAWARD (Hamere)

PersonAward (person_id: INT, award_id: INT, award_year: INT

PersonAward connects a person to an award in a specific year. Including award_year in this relation supports history without duplicating award definitions.

1.6 WORK (Hamere)

Work (work_id: INT, title: VARCHAR (300) , pub_year: INT, url: VARCHAR(400), work_type: (‘paper’, ‘book’, ‘standard’, ‘software’, ‘report’, ‘thesis’, ‘talk’, ‘other’))

Work stores publications, software, or standards associated with influential people. It's separated from person so that a single work can be linked to multiple people and a person can have many works.

1.6.1 PERSON WORK (Hamere)

PersonWork (person_id: INT, work_id: INT)

PersonWork implements the many to many relationship between Person and Work. It also preserves author/contributor order and role which are properties of the relationship rather than of either entity alone.

1.7 NATIONALITY JC

Nationality (nationality_id:INT, name:VARCHAR(100))

The nationality table store information about nationalities, using nationality_id as the primary key and the nationality name.

1.8 USER JC

User(user_id:INT, username:VARCHAR(100), password:VARCHAR(255), access_level:ENIUM)

The user table contains the unique user_id as the primary key. Other attributes include the username, the user's hashed password, and their access_level.

1.9 MANAGEMENT LOG JC

Management_ log(log_id:INT, timestamp:TIMESTAMP, activity_type:VARCHAR(5), details:TEXT, user_id :INT,person_id)

The user_id is a foreign key referencing the User table, indicating which user performed the action.

1.10 AFFILIATIONS

Affiliations (person_id INT, org_id INT, start_year INT (4), end_year INT (4), title VARCHAR (200))

The affiliation contains person_id, and org_id as the primary key from the person and organization entities. Other attributes include the start_year, end_year, and title)

1.11 OtherContributions

OtherContributions(cont_id:INT, person_id:INT)

Other Contributions contains the contribution ID from the contributions table and the person ID from the persons table.

SQL Scrip:

```sql
USE jc45db;

/* Disable foreign key checks to accurately drop all tables */
SET FOREIGN_KEY_CHECKS = 0;

/* Drop any existing tables. Child tables first, then parents */
DROP TABLE IF EXISTS OtherContributions;
DROP TABLE IF EXISTS Contributions;
DROP TABLE IF EXISTS Affiliation;
DROP TABLE IF EXISTS PersonAward;
DROP TABLE IF EXISTS ManagementLog;
DROP TABLE IF EXISTS Works;
DROP TABLE IF EXISTS Award;
DROP TABLE IF EXISTS Person;
DROP TABLE IF EXISTS `Domain`;
DROP TABLE IF EXISTS Nationality;
DROP TABLE IF EXISTS Organization;
DROP TABLE IF EXISTS `User`;
DROP TABLE IF EXISTS Authorship;

/* user_id must be unique and auto-incremented.
   username must be unique; password and access_level NOT NULL. */
CREATE TABLE `User`(
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(100) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    access_level ENUM('admin', 'editor', 'viewer') NOT NULL
);

/* org_id auto-increments, url is unique. */
CREATE TABLE Organization(
    org_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(200) NOT NULL,
    country VARCHAR(50) NOT NULL,
    org_type VARCHAR(50),
    url VARCHAR(300) UNIQUE
);

/* nationality_id auto-increments, name NOT NULL + UNIQUE. */
CREATE TABLE Nationality(
    nationality_id INT PRIMARY KEY AUTO_INCREMENT,
```

```sql
    name VARCHAR(100) UNIQUE NOT NULL
);


/* domain_id auto-increments, name NOT NULL + UNIQUE. */
CREATE TABLE `Domain`(
    domain_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(30) UNIQUE NOT NULL
);


/* person_id auto-increments. active years are 4-digit years. */
CREATE TABLE Person(
    person_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) NOT NULL,
    date_of_birth DATE NOT NULL,
    active_start_year INT CHECK (active_start_year BETWEEN 1000 AND 9999),
    active_end_year   INT CHECK (active_end_year   BETWEEN 1000 AND 9999),
    bio_url VARCHAR(300) UNIQUE,
    domain_id INT,
    nationality_id INT NOT NULL,
    FOREIGN KEY (domain_id)
        REFERENCES Domain(domain_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    FOREIGN KEY (nationality_id)
        REFERENCES Nationality(nationality_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);


/* work_type is an ENUM based on your description. */
CREATE TABLE Works(
    work_id INT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(300) NOT NULL,
    pub_year INT,
    url VARCHAR(300),
    work_type ENUM('paper', 'book', 'standard', 'software',
                   'report', 'thesis', 'talk', 'other') NOT NULL
);
```

```sql
CREATE TABLE Award(
    award_id INT PRIMARY KEY AUTO_INCREMENT,
    award_name VARCHAR(200) NOT NULL,
    category   VARCHAR(100),
    year_awarded INT,
    UNIQUE KEY (award_name)
);

CREATE TABLE PersonAward(
    person_id INT NOT NULL,
    award_id  INT NOT NULL,
    award_year INT,
    PRIMARY KEY (person_id, award_id),
    FOREIGN KEY (person_id) REFERENCES Person(person_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (award_id) REFERENCES Award(award_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);


/* log_id auto-increments. Relates actions done by a User (and optionally about a
Person). */
CREATE TABLE ManagementLog(
    log_id INT PRIMARY KEY AUTO_INCREMENT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    activity_type VARCHAR(50) NOT NULL,
    details TEXT,
    user_id INT NOT NULL,
    person_id INT NULL,
    FOREIGN KEY (user_id) REFERENCES `User`(user_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (person_id) REFERENCES Person(person_id)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);


/* Bridge table for M:N relationship between Person and Organization. */
CREATE TABLE Affiliation(
    person_id INT NOT NULL,
```

```sql
    org_id INT NOT NULL,
    title VARCHAR(100),
    active_start_year INT CHECK (active_start_year BETWEEN 1000 AND 9999),
    active_end_year   INT CHECK (active_end_year   BETWEEN 1000 AND 9999),
    PRIMARY KEY (person_id, org_id),
    FOREIGN KEY (person_id) REFERENCES Person(person_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (org_id) REFERENCES Organization(org_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);


/* cont_id auto-increments; contributions themselves independent. */
CREATE TABLE Contributions(
    cont_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(25) NOT NULL,
    role VARCHAR(20),
    org VARCHAR(20),
    year INT,
    type VARCHAR(20) NOT NULL
);

/* OtherContributions: bridge M:N between Person and Contributions. */
CREATE TABLE OtherContributions(
    person_id INT NOT NULL,
    cont_id INT NOT NULL,
    PRIMARY KEY (person_id, cont_id),
    FOREIGN KEY (person_id) REFERENCES Person(person_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (cont_id) REFERENCES Contributions(cont_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

/* Bridge table for M:N relationship between Person and Works*/
Create TABLE Authorship(
    person_id INT NOT NULL,
    work_id INT NOT NULL,
    PRIMARY KEY (person_id, work_id),
    FOREIGN KEY (person_id) REFERENCES Person(person_id)
```

```
            ON DELETE CASCADE
            ON UPDATE CASCADE,
        FOREIGN KEY (work_id) REFERENCES Works(work_id)
);

/* Re enable FK checks */
SET FOREIGN_KEY_CHECKS = 1;
```

Loading Data:

```
SET FOREIGN_KEY_CHECKS = 0;

-- 1. TRUNCATE (RESET) TABLES
-- This resets the Auto-Increment counters to 1.
TRUNCATE TABLE Authorship;
TRUNCATE TABLE Affiliation;
TRUNCATE TABLE PersonAward;
TRUNCATE TABLE OtherContributions;
TRUNCATE TABLE ManagementLog;
TRUNCATE TABLE Works;
TRUNCATE TABLE Organization;
TRUNCATE TABLE Award;
TRUNCATE TABLE Contributions;
TRUNCATE TABLE Person;
TRUNCATE TABLE Domain;
TRUNCATE TABLE Nationality;
TRUNCATE TABLE User;

SET FOREIGN_KEY_CHECKS = 1;



-- 2. INSERT USERS & LOOKUPS
INSERT INTO User (username, password, access_level) VALUES ('sysadmin',
'hashed_pass', 'admin');

INSERT INTO Nationality (name) VALUES
('Finnish'), ('Hungarian-American'), ('British'), ('American'), ('Dutch'),
('Israeli-American');

INSERT INTO Domain (name) VALUES
```

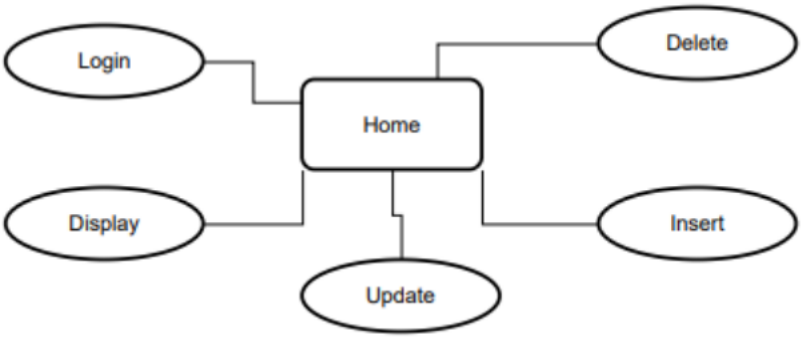
```sql
('Operating Systems'), ('Mathematics'), ('Theoretical Computer Science'),
('Technology'), ('Software Engineering'), ('Programming Languages'), ('Computer
Programming');


-- 3. INSERT ORGANIZATIONS
INSERT INTO Organization (name, country, org_type, url) VALUES
  ('Transmeta Corporation', 'USA', 'Industry', NULL),        -- ID 1
  ('Institute for Advanced Study', 'USA', 'Academic', NULL), -- ID 2
  ('University of Cambridge', 'UK', 'Academic', NULL),       -- ID 3
  ('Microsoft', 'USA', 'Industry', NULL),                    -- ID 4
  ('NASA', 'USA', 'Government', NULL),                       -- ID 5
  ('Python Software Foundation', 'USA', 'Non-profit', NULL), -- ID 6
  ('MIT', 'USA', 'Academic', NULL),                          -- ID 7
  ('NACA', 'USA', 'Government', NULL),                       -- ID 8
  ('Apple Inc.', 'USA', 'Industry', NULL),                   -- ID 9
  ('US Navy', 'USA', 'Government', NULL);                    -- ID 10


-- 4. INSERT PEOPLE
INSERT INTO Person (name, date_of_birth, active_start_year, active_end_year,
bio_url, domain_id, nationality_id) VALUES
  ('Linus Torvalds', '1969-12-28', 1988, 2024, 'http://torvalds.bio', 1, 1),
  ('John von Neumann', '1903-12-28', 1926, 1957, 'http://vonneumann.bio', 2, 2),
  ('Alan Turing', '1912-06-23', 1935, 1954, 'http://turing.bio', 3, 3),
  ('Bill Gates', '1955-10-28', 1975, 2024, 'http://gates.bio', 4, 4),
  ('Margaret Hamilton', '1936-08-17', 1961, 2024, 'http://hamilton.bio', 5, 4),
  ('Guido van Rossum', '1956-01-31', 1989, 2024, 'http://gvr.bio', 6, 5),
  ('Steve Jobs', '1955-02-24', 1976, 2011, 'http://jobs.bio', 4, 4),
  ('Grace Hopper', '1906-12-09', 1944, 1986, 'http://hopper.bio', 7, 4),
  ('Shafi Goldwasser', '1958-06-18', 1983, 2018, 'http://goldwasser.bio', 3, 6),
  ('Katherine Johnson', '1918-08-26', 1953, 1986, 'http://johnson.bio', 2, 4);


-- 5. INSERT WORKS (Using 'Works' Plural)
INSERT INTO Works (title, pub_year, work_type, url) VALUES
  ('Linux Kernel', 1991, 'software', NULL),         -- ID 1
  ('First Draft on EDVAC', 1945, 'report', NULL),   -- ID 2
  ('On Computable Numbers', 1936, 'paper', NULL),   -- ID 3
  ('Microsoft Windows', 1985, 'software', NULL),    -- ID 4
  ('Apollo Guidance Code', 1969, 'software', NULL), -- ID 5
  ('Python Language', 1991, 'software', NULL),      -- ID 6
  ('Macintosh OS', 1984, 'software', NULL),         -- ID 7
```

```sql
  ('Compilers Paper', 1952, 'paper', NULL),         -- ID 8
  ('Lattice Problems', 2002, 'book', NULL),         -- ID 9
  ('Apollo Trajectory', 1969, 'report', NULL);      -- ID 10

-- Link Works (1-to-1 Mapping)
-- TRUNCATE, IDs will be 1-10.
INSERT INTO Authorship (person_id, work_id) VALUES
  (1, 1), (2, 2), (3, 3), (4, 4), (5, 5), (6, 6), (7, 7), (8, 8), (9, 9), (10,
10);



-- 6. INSERT AWARDS
INSERT INTO Award (award_name, category) VALUES
  ('Millennium Tech Prize', 'Tech'),     -- ID 1
  ('Fermi Award', 'Science'),            -- ID 2
  ('Turing Award', 'CS'),                -- ID 3
  ('Medal of Freedom', 'Civilian'),      -- ID 4
  ('NASA Space Act Award', 'Space'),     -- ID 5
  ('Free Software Award', 'CS'),         -- ID 6
  ('National Medal of Tech', 'Tech'),    -- ID 7
  ('Defense Medal', 'Military'),         -- ID 8
  ('Godel Prize', 'CS'),                 -- ID 9
  ('Congressional Gold Medal', 'Civilian'); -- ID 10

INSERT INTO PersonAward (person_id, award_id, award_year) VALUES
  (1, 1, 2012),
  (2, 2, 1956),
  (3, 3, 1966),
  (4, 4, 2016),
  (5, 5, 2003),
  (6, 6, 2001),
  (7, 7, 1985),
  (8, 8, 1986),
  (9, 9, 1993),
  (10, 10, 2019);



-- 7. INSERT AFFILIATIONS
INSERT INTO Affiliation (person_id, org_id, title, active_start_year,
active_end_year) VALUES
  (1, 1, 'Chief Architect', 1997, 2003),
  (2, 2, 'Professor', 1933, 1957),
  (3, 3, 'Fellow', 1935, 1954),
```

```sql
  (4, 4, 'CEO', 1975, 2000),
  (5, 5, 'Director', 1961, 1976),
  (6, 6, 'President', 2001, 2018),
  (7, 9, 'CEO', 1976, 2011),
  (8, 10, 'Rear Admiral', 1943, 1986),
  (9, 7, 'Professor', 1983, 2018),
  (10, 8, 'Mathematician', 1953, 1986);



-- 8. INSERT CONTRIBUTIONS
INSERT INTO Contributions (name, year, type) VALUES
  ('Open Source Movement', 1991, 'other'),          -- ID 1
  ('Game Theory', 1944, 'theory'),                  -- ID 2
  ('Code Breaking', 1940, 'practice'),              -- ID 3
  ('Personal Computing', 1975, 'business'),         -- ID 4
  ('Software Engineering', 1969, 'practice'),       -- ID 5
  ('Readability in Code', 1991, 'theory'),          -- ID 6
  ('Smartphone Revolution', 2007, 'business'),      -- ID 7
  ('COBOL Language', 1959, 'software'),             -- ID 8
  ('Zero-Knowledge Proofs', 1985, 'theory'),        -- ID 9
  ('Orbital Mechanics', 1962, 'math');              -- ID 10

INSERT INTO OtherContributions (person_id, cont_id) VALUES
  (1, 1), (2, 2), (3, 3), (4, 4), (5, 5), (6, 6), (7, 7), (8, 8), (9, 9), (10,
10);
```

IV.User Interface Design:

Website Diagram



Page /File Description:

Website Name/HomePage:

CS Influential People - Database Project pluo.hood.edu/~jc45/index.html

FILE: Database Connection (db.php)

Creates a database connection using PDO to the Mysql server at:

pluto.hood.edu

database #####

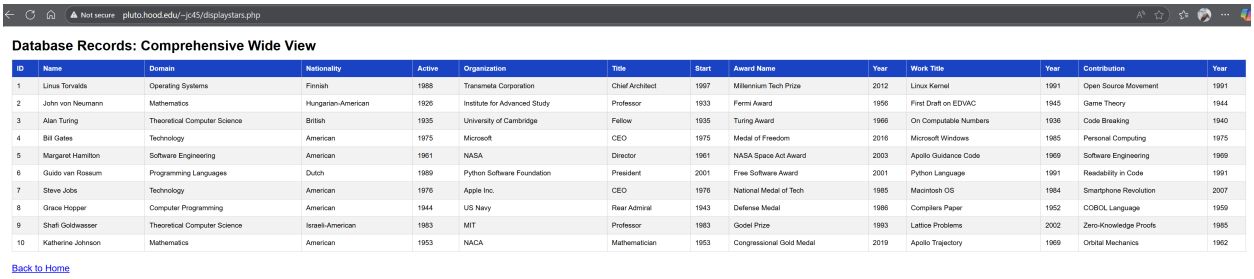
username ####

password ########.

Sets PDO to throw exceptions on errors (PDO::ERRMODE_EXCEPTION). If the connection fails, the script stops immediately and prints: Could **not connect to the database $dbname: <error message>.**

PAGE: Display People (person_display.php)

This each Person in our database, influential person will be presented with information such as domain, nationality, active year person began initial influence, organization, title, start year of title, award name, year award won, work title, year work published, contribution, year contribution publishes.



Not secure pluto.hood.edu/~jc45/displaystars.php

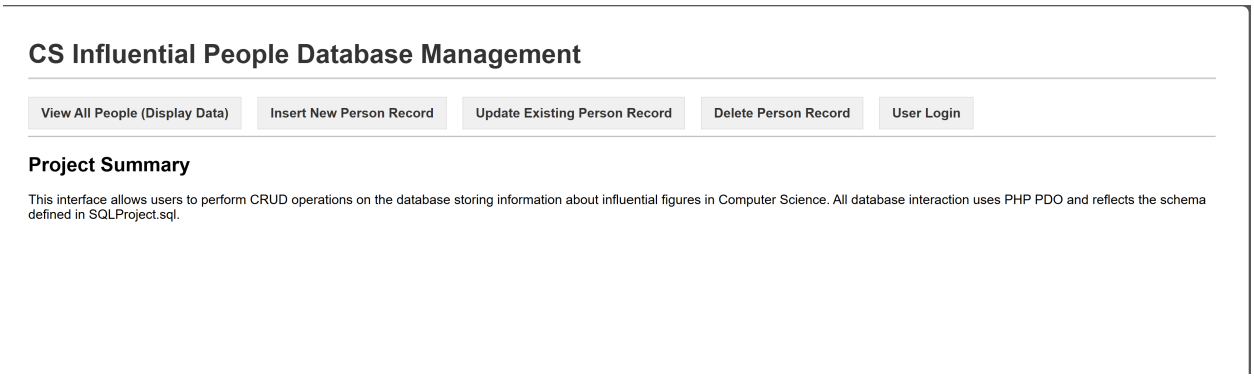
Database Records: Comprehensive Wide View

ID	Name	Domain	Nationality	Active	Organization	Title	Start	Award Name	Year	Work Title	Year	Contribution	Year
1	Linus Torvalds	Operating Systems	Finnish	1988	Transmeta Corporation	Chief Architect	1997	Millennium Tech Prize	2012	Linux Kernel	1991	Open Source Movement	1991
2	John von Neumann	Mathematics	Hungarian-American	1926	Institute for Advanced Study	Professor	1933	Fermi Award	1956	First Draft on EDVAC	1945	Game Theory	1944
3	Alan Turing	Theoretical Computer Science	British	1935	University of Cambridge	Fellow	1935	Turing Award	1966	On Computable Numbers	1936	Code Breaking	1940
4	Bill Gates	Technology	American	1975	Microsoft	CEO	1975	Medal of Freedom	2016	Microsoft Windows	1985	Personal Computing	1975
5	Margaret Hamilton	Software Engineering	American	1961	NASA	Director	1961	NASA Space Act Award	2003	Apollo Guidance Code	1969	Software Engineering	1969
6	Guido van Rossum	Programming Languages	Dutch	1989	Python Software Foundation	President	2001	Free Software Award	2001	Python Language	1991	Readability in Code	1991
7	Steve Jobs	Technology	American	1976	Apple Inc.	CEO	1976	National Medal of Tech	1985	Macintosh OS	1984	Smartphone Revolution	2007
8	Grace Hopper	Computer Programming	American	1944	US Navy	Rear Admiral	1943	Defense Medal	1986	Compilers Paper	1952	COBOL Language	1959
9	Shafi Goldwasser	Theoretical Computer Science	Israeli-American	1983	MIT	Professor	1983	Godel Prize	1993	Lattice Problems	2002	Zero-Knowledge Proofs	1985
10	Katherine Johnson	Mathematics	American	1953	NACA	Mathematician	1953	Congressional Gold Medal	2019	Apollo Trajectory	1969	Orbital Mechanics	1962

Back to Home

PAGE: Home Page (index.html/index.php)

Landing page that serves as the navigation hub for the site. Provides links to all required functions (Display, Insert, Update, Delete, Login) so the website is fully usable starting from one page.



CS Influential People Database Management

View All People (Display Data) | Insert New Person Record | Update Existing Person Record | Delete Person Record | User Login

Project Summary

This interface allows users to perform CRUD operations on the database storing information about influential figures in Computer Science. All database interaction uses PHP PDO and reflects the schema defined in SQLProject.sql.

PAGE Insert PersonForm (insert_person_form.html /insert_person_form.php)

Form page that collects new person information (name, date of borth, years active, bio url, domain_id, nationality_id). Sends the form data using POST to the insert handler.

Insert a New Person

Enter a new person record to insert into the Person table.

Name:
Date of Birth (YYYY-MM-DD): 1969-12-28
Active Start Year:
Active End Year:
Bio URL: http://example.com/bio
Domain ID:
Nationality ID:

Insert Reset

Back to Home

FILE: Insert Person Handler (insert_person.php)

Processes the insert form submission. Validates required fields and inserts a new record into the Person table using a prepared statement, then confirms success and optionally redisplays updated results.

Not Secure | http://pluto.hood.edu/~hs17/Insert_handler.php

1 row added sucessfully!

Proof: Current Person Table

person_id	name	date_of_birth	active_start_year	active_end_year	bio_url	domain_id	nationality_id
1	Linus Torvalds	1969-12-28	1988	2024	http://torvalds.bio	1	1
2	John von Neumann	1903-12-28	1926	1957	http://vonneumann.bio	2	2
3	Alan Turing	1912-06-23	1935	1954	http://turing.bio	3	3
4	Bill Gates	1955-10-28	1975	2024	http://gates.bio	4	4
5	Margaret Hamilton	1936-08-17	1961	2024	http://hamilton.bio	5	4
6	Guido van Rossum	1956-01-31	1989	2024	http://gvr.bio	6	5
7	Steve Jobs	1955-02-24	1976	2011	http://jobs.bio	4	4
8	Grace Hopper	1906-12-09	1944	1986	http://hopper.bio	7	4
9	Shafi Goldwasser	1958-06-18	1983	2018	http://goldwasser.bio	3	6
10	Katherine Johnson	1918-08-26	1953	1986	http://johnson.bio	2	4
11	Mr. John	1975-12-13	1000	1001		2	1

Insert Another | Back to Home

PAGE: Select Person to Update (person_to_update.php)

Shows current person records and provides a simple input field to choose a target person.

Not Secure | http://pluto.hood.edu/~hs17/person_to_update.php

Display the information

person_id	name	date_of_birth	active_start_year	active_end_year	bio_url	domain_id	nationality_id
1	Linus Torvalds	1969-12-28	1988	2024	http://torvalds.bio	1	1
2	John von Neumann	1903-12-28	1926	1957	http://vonneumann.bio	2	2
3	Alan Turing	1912-06-23	1935	1954	http://turing.bio	3	3
4	Bill Gates	1955-10-28	1975	2024	http://gates.bio	4	4
5	Margaret Hamilton	1936-08-17	1961	2024	http://hamilton.bio	5	4
6	Guido van Rossum	1956-01-31	1989	2024	http://gvr.bio	6	5
7	Steve Jobs	1955-02-24	1976	2011	http://jobs.bio	4	4
8	Grace Hopper	1906-12-09	1944	1986	http://hopper.bio	7	4
9	Shafi Goldwasser	1958-06-18	1983	2018	http://goldwasser.bio	3	6
10	Katherine Johnson	1918-08-26	1953	1986	http://johnson.bio	2	4

ID to Select:

ID: [ ]
Select Reset

PAGE: Fill Update Form (fill_to_update_person.php)

Fetches the selected person's current data from the Person table and pre-populates an update form.

PAGE: Update Person Handler (updated_person.php)

Processes the submitted update form and applies changes to the selected person record using a prepared update statement keyed by person_id. Reports the number of rows updated and displays the updated table for confirmation.

Display the information

person_id	name	date_of_birth	active_start_year	active_end_year	bio_url	domain_id	nationality_id
1	Linus Torvalds	1969-12-28	1988	2024	http://torvalds.bio	1	1
2	John von Neumann	1903-12-28	1926	1957	http://vonneumann.bio	2	2
3	Alan Turing	1912-06-23	1935	1954	http://turing.bio	3	3
4	Bill Gates	1955-10-28	1975	2024	http://gates.bio	4	4
5	Margaret Hamilton	1936-08-17	1961	2024	http://hamilton.bio	5	4
6	Guido van Rossum	1956-01-31	1989	2024	http://gvr.bio	6	5
7	Steve Jobs	1955-02-24	1976	2011	http://jobs.bio	4	4
8	Grace Hopper	1906-12-09	1944	1986	http://hopper.bio	7	4
9	Shafi Goldwasser	1958-06-18	1983	2018	http://goldwasser.bio	3	6
10	Katherine Johnson	1918-08-26	1953	1986	http://johnson.bio	2	4

ID to Select:

ID: 4
Select Reset

Name: William Bill Gates
Date of Birth (YYYY-MM-DD): 1955-10-28
Active Start Year: 1975
Active End Year: 2024
Bio URL: http://gates.bio
Domain ID: 4
Nationality ID: 4

Update Reset

1 row updated sucessfully!

person_id	name	date_of_birth	active_start_year	active_end_year	bio_url	domain_id	nationality_id
1	Linus Torvalds	1969-12-28	1988	2024	http://torvalds.bio	1	1
2	John von Neumann	1903-12-28	1926	1957	http://vonneumann.bio	2	2
3	Alan Turing	1912-06-23	1935	1954	http://turing.bio	3	3
4	William Bill Gates	1955-10-28	1975	2024	http://gates.bio	4	4
5	Margaret Hamilton	1936-08-17	1961	2024	http://hamilton.bio	5	4
6	Guido van Rossum	1956-01-31	1989	2024	http://gvr.bio	6	5
7	Steve Jobs	1955-02-24	1976	2011	http://jobs.bio	4	4
8	Grace Hopper	1906-12-09	1944	1986	http://hopper.bio	7	4
9	Shafi Goldwasser	1958-06-18	1983	2018	http://goldwasser.bio	3	6
10	Katherine Johnson	1918-08-26	1953	1986	http://johnson.bio	2	4

PAGE: Delete Person Page (updated_person_delete.php)

Provides a delete interface so the user can choose which record to delete. Submits the selected person to the delete handler.

PAGE: Delete Person Handler (person_delete.php)

Deletes the selected person record from the person table using a prepared delete statement. Displays a confirmation messgae and a refreshed list/table to verify the deletion.